Reference count one at the write

Gavin Mendel-Gleason

Abstract

A reference-counted runtime updates an object in place exactly when its count is one. Implementations decide this at run time, because one compiled body serves owning and borrowing call sites; but the count is determined by which references the term and the surrounding structure hold, so it is a static property of the program. This paper formalises a pure core, a reference-counted target over a counted heap, and the lowering between them, and proves the pair that characterises the target's write: an owned store is destructive and keeps its location, a shared store copies. Everything is mechanised in Lean 4 with no library dependency. The lowering is proved total and deterministic, not correct; that is stated.

1 The guarantee

The write is destructive when its operand is owned once and nothing else names the cell; otherwise it copies. That is the pair proved here, and it is the only claim made about the target's write.

```
theorem store_owned_is_destructive {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {y : Var} {r : Ref} {i : Nat} {v : Expr} {ks : List Ref} (hc : Couple c hs ρ m)
    (hρ : ρ.lookup y = some r) (hown : Owns ρ m (.store (.var y) i v))
    (hm : (expand m).count r = 1) (hslot : (CHeap.slots c).count r = 0)
    (hrun : hs.count r = 0) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup r = some ks

theorem store_shared_copies {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {y : Var} {r : Ref} {i : Nat} {v : Expr} {ks : List Ref} (hc : Couple c hs ρ m)
    (hρ : ρ.lookup y = some r) (hown : Owns ρ m (.store (.var y) i v))
    (hm : 2 ≤ (expand m).count r ∨ 1 ≤ (CHeap.slots c).count r) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup (fresh c ks) = some ks ∧
      (update c (hs ++ expand m) r ks).1.lookup r = CHeap.lookup c r ∧
      (update c (hs ++ expand m) r ks).1.count r = CHeap.count c r - 1
```

`Couple` is the coupling invariant (§6), `Owns` is the term-indexed account of the term's own references, `expand m` is the reference list the account stands for, and `update` is the heap's two-branch write (§2). The two hypotheses the account cannot supply — no slot and no run handle naming the cell — are named rather than hidden (§6.1). Read together the two theorems say: the write is in place exactly where the account counts the operand once.

2 The counted heap

A cell holds the references its slots name and the count the runtime keeps for it; a heap lists the live cells.

```
structure Cell where
  slots : List Ref
  count : Nat
deriving DecidableEq, Repr

abbrev CHeap := List (Ref × Cell)
```

One reference names one cell, so the count a heap keeps at a reference is not a choice among cells.

```
def CHeap.WF (c : CHeap) : Prop := (CHeap.refs c).Nodup
```

No live cell has count zero, because a release deallocates at zero.

```
def CHeap.Live (c : CHeap) : Prop := c.find? (fun e => e.2.count == 0) = none
```

A cell's count is what it must be: its occurrences among the live cells' slots, plus the handles held outside the heap — the references the running term holds.

```
def Counted (c : CHeap) (handles : List Ref) : Prop :=
  ∀ r, CHeap.count c r = refCount (CHeap.presentation c) handles r
```

The write branches on that count: at one the cell keeps its location (and the references its old slots held are dropped), otherwise a fresh cell takes the written slots and the old cell loses a reference.

```
def update (c : CHeap) (handles : List Ref) (r : Ref) (ks : List Ref) : CHeap × List
    Ref :=
  if CHeap.count c r = 1 then
    (CHeap.cascade (CHeap.writeSlots c r ks), handles)
  else
    ((fresh c ks, cellOf ks) :: CHeap.addOccurrences (CHeap.adjust c r (· - 1)) ks,
      fresh c ks :: dropHandle handles r)
```

The two branches, as already proved of the heap: a destructive write is read back at the same location, a copying write at a location the heap did not list.

```
theorem update_preserves_location_of_exclusive {c : CHeap} {handles : List Ref}
    (hW : CHeap.WF c) (hc : Counted c handles) {r : Ref} {ks : List Ref}
    (h₁ : Exclusive (CHeap.presentation c) handles r) (hhandle : 1 ≤ handles.count r)
    (hks : r ∉ ks) :
    (update c handles r ks).1.lookup r = some ks

theorem update_copies_of_shared {c : CHeap} {handles : List Ref} (hc : Counted c
    handles)
    {r : Ref} {ks : List Ref} (h₂ : ¬ Exclusive (CHeap.presentation c) handles r)
    (hhandle : 1 ≤ handles.count r) (hks : r ∉ ks) :
    (update c handles r ks).1.lookup (fresh c ks) = some ks ∧
      (update c handles r ks).1.lookup r = CHeap.lookup c r ∧
      (update c handles r ks).1.count r = CHeap.count c r - 1 ∧
      fresh c ks ∉ CHeap.refs c
```

3 The source calculus

One term type serves both calculi. Nodes are nullary, unary or binary; an n-ary node is a spine. The write is a subject, a slot index and a value.

```
inductive Expr where
  | var (x : Var)
  | lam (x : Var) (body : Expr)
  | app (f a : Expr)
  | bind (x : Var) (e₁ e₂ : Expr)
  | matchC (s : Expr) (c : Ctor) (ys : List Var) (body : Expr)
  | dup (x : Var) (e : Expr)
  | drop (x : Var) (e : Expr)
  | con0 (c : Ctor)
  | con1 (c : Ctor) (a : Expr)
  | con2 (c : Ctor) (a b : Expr)
  | store (s : Expr) (i : Nat) (v : Expr)
deriving DecidableEq, Repr
```

The source is the fragment with neither instruction; the free-variable function and the closedness discipline are what make the substitution below sound.

```
inductive Pure : Expr → Prop where
  | var (x : Var) : Pure (.var x)
  | lam (x : Var) (body : Expr) : Pure body → Pure (.lam x body)
  | app (f a : Expr) : Pure f → Pure a → Pure (.app f a)
  | bind (x : Var) (e₁ e₂ : Expr) : Pure e₁ → Pure e₂ → Pure (.bind x e₁ e₂)
  | matchC (s : Expr) (c : Ctor) (ys : List Var) (body : Expr) :
      Pure s → Pure body → Pure (.matchC s c ys body)
  | con0 (c : Ctor) : Pure (.con0 c)
  | con1 (c : Ctor) (a : Expr) : Pure a → Pure (.con1 c a)
  | con2 (c : Ctor) (a b : Expr) : Pure a → Pure b → Pure (.con2 c a b)
  | store (s : Expr) (i : Nat) (v : Expr) : Pure s → Pure v → Pure (.store s i v)

def uses : Expr → List Var
  | .var x => [x]
  | .lam x body => (uses body).filter (fun y => y ≠ x)
  | .app f a => uses f ++ uses a
  | .bind x e₁ e₂ => uses e₁ ++ (uses e₂).filter (fun y => y ≠ x)
  | .matchC s _ ys body => uses s ++ (uses body).filter (fun y => y ∉ ys)
  | .dup x e => x :: uses e
  | .drop x e => x :: uses e
  | .con0 _ => []
  | .con1 _ a => uses a
  | .con2 _ a b => uses a ++ uses b
  | .store s _ v => uses s ++ uses v

def Closed (e : Expr) : Prop := uses e = []
```

Values are the canonical forms the reduction's rules match on.

```
inductive Value : Expr → Prop where
  | var (x : Var) : Value (.var x)
  | lam (x : Var) (body : Expr) : Value (.lam x body)
```

```
  | con0 (c : Ctor) : Value (.con0 c)
  | con1 {c : Ctor} {a : Expr} (h : Value a) : Value (.con1 c a)
  | con2 {c : Ctor} {a b : Expr} (h₁ : Value a) (h₂ : Value b) : Value (.con2 c a b)
```

The redexes: application of an abstraction, the two binders, a match on a value node, and the write read as a functional update.

```
inductive Root : Expr → Expr → Prop where
  | beta {x : Var} {body v : Expr} :
      Value v → Root (.app (.lam x body) v) (subst x v body)
  | bindVal {x : Var} {v body : Expr} :
      Value v → Root (.bind x v body) (subst x v body)
  | matchVal {s : Expr} {c : Ctor} {ys : List Var} {body : Expr} : Value s →
      nodeTag? s = some c → ys.length = (nodeFields s).length →
      Root (.matchC s c ys body) (substAll ys (nodeFields s) body)
  | storeVal {s : Expr} {i : Nat} {v n' : Expr} : Value s → Value v →
      slotUpdate s i v = some n' → Root (.store s i v) n'
```

The reduction is a redex, or a step inside one of the contexts below. There is no rule inside a match's subject, a write's subject or a node's fields: the source is written so that those are values already.

```
inductive Step : Expr → Expr → Prop where
  | root {e e' : Expr} : Root e e' → Step e e'
  | appFun {f f' a : Expr} : Step f f' → Step (.app f a) (.app f' a)
  | appArg {f a a' : Expr} : Value f → Step a a' → Step (.app f a) (.app f a')
  | bindFun {e₁ e₁' : Expr} {x : Var} {e₂ : Expr} : Step e₁ e₁' →
      Step (.bind x e₁ e₂) (.bind x e₁' e₂)
```

The contexts the source steps in.

```
inductive Ectx where
  | hole
  | appL (E : Ectx) (a : Expr)
  | appR (f : Expr) (E : Ectx)
  | bindL (x : Var) (E : Ectx) (e₂ : Expr)
```

Substitution is naive, and therefore capture free only on closed arguments: this is the discipline the source is used at.

```
theorem uses_subst_of_closed : ∀ (e : Expr) (x : Var) (v : Expr), Pure e → Closed v →
    uses (subst x v e) = (uses e).filter (fun y => y ≠ x)

theorem step_preserves_closed {e e' : Expr} (hs : Step e e') : Pure e → Closed e →
    Closed e'
```

The reduction is a function, which is what lets the lowering be one too.

```
theorem step_deterministic : ∀ {e e₁ : Expr}, Step e e₁ → ∀ {e₂ : Expr}, Step e e₂ → e
    ₁ = e₂
```

4 The lowering

A derivation relates a source term to its target under a borrowed and an owned multiset of occurrences. Availability is about names, not multiplicities: a name used twice may be fitting at one owned occurrence, and the strategy supplies the second use with a duplication.

```
def Fits (Δ Γ : Occs) (e : Expr) : Prop := ∀ x ∈ uses e, x ∈ Γ ∨ x ∈ Δ
```

The relation: one rule per source construct. The splits are the functions `claimEnv`, `bindEnv` and `armEnv` rather than an existential, and each rule's conclusion wraps its translation in `wrap`, the prologue that duplicates and releases.

```
inductive Lower : Occs → Occs → Expr → Expr → Prop where
  | var {Δ Γ x} (h : x ∈ Γ ∨ x ∈ Δ) :
      Lower Δ Γ (.var x) (wrap Γ [x] (.var x))
  | lam {Δ Γ x body body'} (h : Lower (forget Δ x) (bindEnv Γ x (uses body)) body
    body') :
      Lower Δ Γ (.lam x body) (wrap Γ (uses (.lam x body)) (.lam x body'))
  | app {Δ Γ f a f' a'} (hf : Lower Δ (claimEnv Γ (uses f)) f f')
      (ha : Lower Δ (claimEnv Γ (uses a)) a a') :
      Lower Δ Γ (.app f a) (wrap Γ (uses (.app f a)) (.app f' a'))
  | bind {Δ Γ x e₁ e₂ e₁' e₂'} (h₁ : Lower Δ (claimEnv Γ (uses e₁)) e₁ e₁')
      (h₂ : Lower (forget Δ x) (bindEnv Γ x (uses e₂)) e₂ e₂') :
      Lower Δ Γ (.bind x e₁ e₂) (wrap Γ (uses (.bind x e₁ e₂)) (.bind x e₁' e₂'))
  | matchC {Δ Γ s c ys body s' body'} (hs : Lower Δ (claimEnv Γ (uses s)) s s')
      (h : Lower (forgetAll Δ ys) (armEnv Γ ys (uses body)) body body') :
      Lower Δ Γ (.matchC s c ys body)
        (wrap Γ (uses (.matchC s c ys body)) (.matchC s' c ys body'))
  | con0 {Δ Γ c} : Lower Δ Γ (.con0 c) (wrap Γ [] (.con0 c))
  | con1 {Δ Γ c a a'} (h : Lower Δ (claimEnv Γ (uses a)) a a') :
      Lower Δ Γ (.con1 c a) (wrap Γ (uses (.con1 c a)) (.con1 c a'))
  | con2 {Δ Γ c a b a' b'} (ha : Lower Δ (claimEnv Γ (uses a)) a a')
      (hb : Lower Δ (claimEnv Γ (uses b)) b b') :
      Lower Δ Γ (.con2 c a b) (wrap Γ (uses (.con2 c a b)) (.con2 c a' b'))
  | store {Δ Γ s i v s' v'} (hs : Lower Δ (claimEnv Γ (uses s)) s s')
      (hv : Lower Δ (claimEnv Γ (uses v)) v v') :
      Lower Δ Γ (.store s i v)
        (wrap Γ (uses (.store s i v)) (.store s' i v'))
```

The relation is exactly the graph of the computable strategy `lowerB` — `lowerB_sound` and `lowerB_complete` are its two directions — so the strategy's own properties are the relation's. It is total on the fragment and deterministic, and the second is why the splits are canonical: with an arbitrary split of a binding's environment the one required duplication could be placed in either subterm and the targets would differ.

```
theorem lower_total {Δ Γ : Occs} {e : Expr} (hp : Pure e) (hfit : Fits Δ Γ e) :
    ∃ e', Lower Δ Γ e e'

theorem lower_deterministic {Δ Γ : Occs} {e e₁ e₂ : Expr} (h₁ : Lower Δ Γ e e₁)
    (h₂ : Lower Δ Γ e e₂) : e₁ = e₂
```

These are properties of the translation. Nothing here says the target computes what the source computes; that is §8.

5 The target and its runtime

A configuration carries the heap, the handles the run holds, an environment from variables to references, the term's own account of its references, and the term.

```
abbrev TEnv := List (Var × Ref)
abbrev Multi := List (Var × Ref)
def expand (m : Multi) : List Ref := m.map (fun p => p.2)
```

The account is one pair per occurrence the term itself holds. Each instruction moves it in step with the heap: a duplication increments the count and adds an occurrence; a release decrements, or deallocates at a count of one; a binding or application of a variable moves an occurrence between names; a match binds the arm's names to the cell's slots; and the write is the heap's own `update`, given the account as outside handles in addition to the run's.

```
inductive TStep :
    CHeap → List Ref → TEnv → Multi → Expr → CHeap → List Ref → TEnv → Multi →
    Expr → Prop where
  | dup {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {r : Ref} {e :
    Expr}
      (hρ : ρ.lookup y = some r) (hlive : r ∈ CHeap.refs c) :
      TStep c hs ρ m (.dup y e) (dup c hs r).1 hs ρ (Multi.insert m y r) e
  | dropShared {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {r : Ref}
      {e : Expr} (hρ : ρ.lookup y = some r) (h₂ : 2 ≤ CHeap.count c r)
      (hsub : (y, r) ∈ m) :
      TStep c hs ρ m (.drop y e) (decRef c hs r).1 hs ρ (Multi.dropOcc m y r) e
  | dropFinal {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {r : Ref}
      {e : Expr} (hρ : ρ.lookup y = some r) (h₁ : CHeap.count c r = 1)
      (hsub : (y, r) ∈ m) :
      TStep c hs ρ m (.drop y e) (release c hs r).1 hs ρ (Multi.dropOcc m y r) e
  | bindVar {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {x z : Var} {e₂ : Expr}
      {r : Ref} (hρ : ρ.lookup z = some r) (hsub : (z, r) ∈ m)
      (hhigh : nameOnly m x r = true) :
      TStep c hs ρ m (.bind x (.var z) e₂) c hs (TEnv.set ρ x r)
        (Multi.insert (Multi.dropOcc m z r) x r) e₂
  | bindVal {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {x : Var} {v e₂ : Expr}
      {c' : CHeap} {hs' : List Ref} {r : Ref} (hv : ValRef c hs ρ v c' hs' r)
      (halloc : ∀ z, v ≠ .var z) (hfree : freeName m x = true) :
      TStep c hs ρ m (.bind x v e₂) c' (dropHandle hs' r) (TEnv.set ρ x r)
        (Multi.insert m x r) e₂
  | appVar {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {x z : Var} {body : Expr}
      {r : Ref} (hρ : ρ.lookup z = some r) (hsub : (z, r) ∈ m)
      (hhigh : nameOnly m x r = true) :
      TStep c hs ρ m (.app (.lam x body) (.var z)) c hs (TEnv.set ρ x r)
        (Multi.insert (Multi.dropOcc m z r) x r) body
  | appVal {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {x : Var} {body v : Expr}
      {c' : CHeap} {hs' : List Ref} {r : Ref} (hv : ValRef c hs ρ v c' hs' r)
      (halloc : ∀ z, v ≠ .var z) (hfree : freeName m x = true) :
      TStep c hs ρ m (.app (.lam x body) v) c' (dropHandle hs' r) (TEnv.set ρ x r)
        (Multi.insert m x r) body
  | matchC {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {z : Var} {ctag : Ctor}
      {ys : List Var} {body : Expr} {r : Ref} (hρ : ρ.lookup z = some r)
      (hlen : ys.length = (CHeap.slotsAt c r).length) (hfree : ∀ p ∈ m, p.1 ∉ ys) :
```

```
      TStep c hs ρ m (.matchC (.var z) ctag ys body) c hs
        (TEnv.bindAll ρ ys (CHeap.slotsAt c r)) m body
  | storeExclusive {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {i :
    Nat}
      {v : Expr} {c1 : CHeap} {hs1 : List Ref} {wv r : Ref} (hρ : ρ.lookup y = some r)
      (hv : ValRef c hs ρ v c1 hs1 wv) (hi : i < (CHeap.slotsAt c1 r).length)
      (hchk : writeCheck c1 r (storeSlots c1 r i wv) = true) (h₁ : CHeap.count c1 r =
    1)
      (hsub : (y, r) ∈ m) :
      TStep c hs ρ m (.store (.var y) i v)
        (update c1 (hs1 ++ expand m) r (storeSlots c1 r i wv)).1 hs1 ρ m (.var y)
  | storeShared {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {i : Nat}
      {v : Expr} {c1 : CHeap} {hs1 : List Ref} {wv r : Ref} (hρ : ρ.lookup y = some r)
      (hv : ValRef c hs ρ v c1 hs1 wv) (hi : i < (CHeap.slotsAt c1 r).length)
      (hchk : writeCheck c1 r (storeSlots c1 r i wv) = true) (hsub : (y, r) ∈ m)
      (hgone : nameGone m y r = true) (h₂ : CHeap.count c1 r ≠ 1) :
      TStep c hs ρ m (.store (.var y) i v)
        (update c1 (hs1 ++ expand m) r (storeSlots c1 r i wv)).1 hs1
        (TEnv.set ρ y (fresh c1 (storeSlots c1 r i wv)))
```

The two store rules are the two branches of `update`, and the only difference between them is the count. Each reads the subject's occurrence out of the term's own account (`hsub`), so an account that does not hold it cannot step.

The positions the runtime steps in: a step inside a well-formed context is a step. The rules above are the steps at a configuration's root; the remaining rules of `TStep` — the function position of an application, its argument while the function is a value, a binding's right-hand side, and a node's fields — are these contexts one by one, which `tstep_of_plug` states.

```
inductive TCtx where
  | hole
  | appL (E : TCtx) (a : Expr)
  | appR (f : Expr) (E : TCtx)
  | bindL (x : Var) (E : TCtx) (e₂ : Expr)
  | con1L (ct : Ctor) (E : TCtx)
  | con2L (ct : Ctor) (E : TCtx) (b : Expr)
  | con2R (ct : Ctor) (a : Expr) (E : TCtx)

theorem tstep_of_plug : ∀ {E : TCtx} {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {r e' : Expr} {c' : CHeap} {hs' : List Ref} {ρ' : TEnv} {m' : Multi},
    TCtx.WF E → TStep c hs ρ m r c' hs' ρ' m' e' →
      TStep c hs ρ m (E.plug r) c' hs' ρ' m' (E.plug e')
```

6 The account, and the pair

Exact tagging: each pair names the reference its variable is bound to, not merely some reference. The weaker reading admits a mis-tagged pair and makes the theorems below arithmetic.

```
def Bound (ρ : TEnv) (m : Multi) : Prop := ∀ p ∈ m, ρ.lookup p.1 = some p.2
```

The account is of the term: each consuming instruction reads its occurrence out of it, and a store reads the occurrence its subject names.

```
inductive Owns (ρ : TEnv) : Multi → Expr → Prop where
  | drop {m : Multi} {y : Var} {r : Ref} {e : Expr} (hρ : ρ.lookup y = some r)
      (hmem : (y, r) ∈ m) : Owns ρ m (.drop y e)
  | bindVar {m : Multi} {x z : Var} {r : Ref} {e₂ : Expr} (hρ : ρ.lookup z = some r)
      (hmem : (z, r) ∈ m) (hheld : ∀ p ∈ m, p.1 = x → p.2 = r) :
      Owns ρ m (.bind x (.var z) e₂)
  | appVar {m : Multi} {x z : Var} {r : Ref} {body : Expr} (hρ : ρ.lookup z = some r)
      (hmem : (z, r) ∈ m) (hheld : ∀ p ∈ m, p.1 = x → p.2 = r) :
      Owns ρ m (.app (.lam x body) (.var z))
  | store {m : Multi} {y : Var} {r : Ref} {i : Nat} {v : Expr} (hρ : ρ.lookup y = some
    r)
      (hmem : (y, r) ∈ m) : Owns ρ m (.store (.var y) i v)
```

The coupling invariant: the term's own references are handles. That is the whole content of the middle conjunct.

```
def Couple (c : CHeap) (hs : List Ref) (ρ : TEnv) (m : Multi) : Prop :=
  CHeap.WF c ∧ Counted c (hs ++ expand m) ∧ Bound ρ m
```

Every step preserves it, contexts included.

```
theorem couple_step {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {e : Expr}
    {c' : CHeap} {hs' : List Ref} {ρ' : TEnv} {m' : Multi} {e' : Expr}
    (h : TStep c hs ρ m e c' hs' ρ' m' e') (hc : Couple c hs ρ m) :
    Couple c' hs' ρ' m'
```

6.1 The two hypotheses, and where they come from

`store_owned_is_destructive` cannot derive, from the account, that no heap slot and no run handle names the cell. Both are what a nested update's take-out provides: the element is removed from its parent's slots, and the pair the take-out hands back is destructured and its handle released before the focus is checked. The mechanisation names them in the theorem and cites the take-out lemmas that supply them.

The second theorem is what keeps the first from being vacuous: a subject a second reference names *must* copy, or the update would be visible through the alias. A statement that every store is destructive is simply false, and the pair is the honest form of the claim.

7 Discussion

The guarantee is static although the runtime tests. The count at a write is the term's own occurrences plus the structure's: a function of the term, the heap and the environment, and not a value read from a run. A test emitted by a compiler is a residual of compiling one body for two ownership situations. A program whose ownership is known does not need the test to be proved about it; only executed.

What is trusted. Three things. The take-out's contribution: that a nested update's cell has no slot and no handle naming it is a property of the nesting, and is taken here as the two named hypotheses. The compiler's lowering: that a compiler turns a language's write paths into the shape modelled here is not proved; in the accompanying repository that link is *checked* by reading the post-reference-counting intermediate representation the pinned toolchain emits and asserting the

shape on one executing path, failing the build when it does not hold. The runtime's primitive: the branch that stores in place is a runtime test whose meaning is documented in the runtime and not modelled here.

What is not proved. The lowering's semantic correctness: no theorem says the target computes what the source computes, or that it is garbage free. What is proved of the lowering is totality and determinism. The declarative relation of the paper this follows is not the one formalised: the strategy's splits are canonical, for determinism, so the results are properties of *this* strategy, which is what a compiler implements.

8 Mechanisation, and related work

The development is roughly 5000 lines of Lean 4 across five modules against the core library only, with no mathematics library dependency. There is no `sorry`, no evaluation by native decision, and every representative theorem's axiom list is asserted, so a proof that acquired a foreign axiom fails the build. Two design facts the formalisation forced are recorded as toolchain facts: terms are a flat inductive, because a nested one cannot be inducted on or derive decidable equality; and an evaluated check needs a structural recursion, because the kernel does not reduce a well-founded one.

Reference counting with reuse. In-place update of a shared value, and the precise counting with ownership that makes it possible, is the design of Lean 4's runtime [1] and is formalised as the linear resource calculus λ^1 in the PeTEX paper [2], which proves the counting sound and garbage free. This development follows that shape and differs where this paper's subject differs: the deliverable is the count at the write, as a pair, with the hypotheses the account cannot supply named; and the translation is a canonical strategy rather than the declarative relation.

Linearity, and formal models of counting. An ordinary type system does not track uniqueness: Linear Haskell [3] gives a linear arrow, and its guarantee is about the use of a value, not about a count in a heap. A formal model of reference counting for code generation appears in Férey and Shankar [4]. Neither is a count-one guarantee for a write.

Future work. Three steps: the lowering's semantic correctness (result agreement and garbage freedom); the surface encoding that would make "every write in an accepted program is destructive" a statement about a client's programs rather than about a strategy; and a verified account, or a soundness theorem for the check, of the compiler's pass.

References

[1] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: reference counting optimized for purely functional programming. In *Proceedings of the 31st Symposium on Implementation and Application of Functional Languages (IFL '19)*, 2019. arXiv:1908.05647.

[2] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. PeTEX: garbage free reference counting with reuse. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*, 2021. Extended version: Microsoft Research Technical Report MSR-TR-2020-42, which contains the proofs.

[3] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear Haskell: practical linearity in a higher-order polymorphic language. In *Proceedings of the 45th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '18)*, 2018.

[4] Gaspard Férey and Natarajan Shankar. Code generation using a formal model of reference counting. In *NASA Formal Methods (NFM 2016)*, pages 150–165, 2016.

[5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction (CADE-28)*, 2021.

[6] Gavin Mendel-Gleason. The mechanisation of this paper, in the Scidonia `lean` repository: `Scidonia/RC/` and `Scidonia/FracPerm/StateVar/RCHeap.lean`, with the checks in `ScidoniaTest/RC.lean` and `ScidoniaTest/Target.lean`. `https://github.com/scidonia/lean-examples`.